Modularité

Les bonnes pratiques de programmation :

Un grand programme demande une organisation, un découpage en parties bien identifiées et bien documentées, et indépendantes au mieux (réutilisables dans d'autres programmes).

I) UN CAS CONCRET : RECHERCHE DE DOUBLONS DANS UN TABLEAU :

Existe-t-il deux personnes dans un groupe ayant la même date d'anniversaire ? (Paradoxe des anniversaires : dans un groupe de 23 personnes au moins, il y a au moins une chance sur deux que deux personnes aient la même date d'anniversaire !).
Dans toute la suite, pour simplifier, une date anniversaire est représentée par un nombre de 1 à 366.

Solution 1 :

```
Créer un tableau secondaire vide S
Pour chaque élément du tableau T :
        tester si l'élément est déjà dans le tableau secondaire S. Si oui, renvoyer True
        ajouter cet élément au tableau secondaire
Renvoyer False
```

Inconvénient : non classé donc coût : parcours du tableau S à chaque nouvel élément de T. $O(n^2)$.

A faire :
Implémenter cet algorithme.

Solution 2 :
Créer un tableau de la taille fixe des possibles des valeurs de T, à False partout (pour les anniversaires : taille 366 dates possibles). Placer les éléments de T à leur rang. Avantage : classé donc comparaison quasi instantanée.

```
Créer un tableau secondaire S entièrement à False
Pour chaque élément x du tableau T :
        tester si S[x] est à True : Si oui, renvoyer True
        mettre S[x] à True
Renvoyer False
```

Coût : faible, l'accès à S[x] est instantané, mais la place mémoire est celle de S (alors que T ne possède que 23 éléments dans notre exemple).
Inconvénient : place mémoire.

A faire :
Implémenter cet algorithme.

Solution 3 : = solution 2 où S n'est plus un tableau, mais un un nombre binaire où les 1 représentent True et les 0 False (appelé tableau de bits (*bit array*)).
Coût faible, place mémoire bien inférieure à celle d'un tableau.
Le nombre s est représenté sur 366 bits, quand même !

Astuces et rappels : opérations logiques sur les bits.
Le nombre s sera formé de 366 bits, à 1 si True, à 0 si False. C'est toujours moins gros en mémoire qu'un tableau de 366 cases.
Pour tester si tel élément x a déjà été vu dans le tableau, il faut lire le bit de rang x sur le nombre s.
Les opérations logiques nous y aident : un & (bit à bit, and) entre s et 2^x est ce qu'il nous faut.
Pour mettre à 1 le bit de rang x ; on fait un | (bit à bit, or) entre s et 2^x.

2^x s'écrit 2 ** x ou par x décalage à gauche des bits du nombre 1 : 1<<x.
L'opérateur << effectuera un "décalage vers la gauche" au niveau du bit, la valeur de l'opérande gauche étant déplacée par le nombre de bits fournis par l'opérande droit.

```
# 2 = 0b10
2 << 2
# Out: 8
# 8 = 0b1000

bin(2 << 2)
# Out: 0b1000
```

L'exécution d'un décalage de 1 gauche équivaut à une multiplication par 2 :

```
7 << 1
# Out: 14
```

Effectuer un décalage de n de gauche équivaut à une multiplication par 2**n :

```
3 << 4
# Out: 48
```

Fonctionne avec la représentation décimale. C'est très pratique.

<u>A faire</u> :
Implémenter cet algorithme.

<u>Solution 4</u> : en réalité, la taille des mots est limitée à 64 bits par exemple.
On fait un tableau de nombres de 64 bits. Donc pour 366, un tableau de 6 nombres suffit.
Le nombre x = i + 64 j (décomposition en poids de 64 + reste) est représenté par un bit à 1 au rang i sur l'entier à la place j dans le tableau.

Entier j = 5	Entier j = 4	Entier j = 3	Entier j = 2	Entier j = 1	Entier j = 0
64 bits	64 bits	64 bits	64 bits	64 bits	64 bits

Bit 7 à 1 dans cette case

Ex : x = $263 = 4 \times 64 + 7$

j = x//64 = 4, i = x % 64 = 7
Conditions pour tester présence : **s[j] & 2^i non nul** : peut s'écrire s[x//64] & (1 << x%64) != 0

<u>A faire</u> :
Implémenter cet algorithme.

<u>Solution mixte</u> : approche de la table de hachage.
= tableau court (taille du tableau à sonder) avec éléments classés, et non pas placés à la fin.
Un élément x est placé dans la case x%23 (reste de la division).
Plusieurs éléments peuvent donc arriver dans le même cas, (mais peu, statistiquement). Mais il est très rapide de tester l'égalité dans une case ciblée (pas besoin de parcourir tout le tableau).
En résumé

Solution 1 : petit tableau secondaire
Solution 2 : grand tableau secondaire
Solution 3 : bit array
Solution 4 : bit array

Factorisation du code.
Déjà quatre solutions pour un même problème ! **Distinguer fonction et implémentation**.
On remarque que ces programmes ont une partie commune indépendantes des solutions implémentées pour les fonctions spécifiques :

```
# Recherche d'existence de doublons dans un tableau
```

```
def doublons1(tableau):
    '''Renvoie True si le tableau contient un doublon.
    Méthode ...
    '''
    s = ...                    ← cree( )
        for x in tableau:
            if … :             ← contient(s,x)
                return True
            ...                ← ajoute(s,x)
    return False
```

Idée : séparer le code commun et délocaliser les fonctions spécifiques : factorisation, séparation des rôles des programmeurs, transparence pour l'utilisateur, réutilisation ailleurs …

Vu sous cet angle, on parle d'**interface d'un programme**, utilisant des **modules**.

<u>A faire</u> :
Implémenter ce programme en le séparant en programme principal et en un module qui devra contenir les fonctions cree, contient, et ajoute.

Interface (pas au sens graphique, ici) : **ensemble des fonctions** auxquelles le programmeur ou l'utilisateur a recours **dans le programme principal, avec leur documentation**.
C'est aussi un **contrat** (garantie de résultats) avec l'utilisateur, quel que soit le moyen choisi.
Un module peut être réutilisé pour d'autres programmes principaux.

Interface du programme principal de recherche de doublons	
cree( )	crée une structure de dates vides
contient(s, x)	renvoie True si la date x est dans s
ajoute(s, x)	ajoute la date x à la structure s

II) <u>NOTION D'ENCAPSULATION</u> :

L'auteur d'un module peut travailler indépendamment avec ses propres solutions, mais sa solution doit donner les résultats annoncés par le contrat interface.
L'auteur d'un module peut donc programmer ses propres **fonctions à usage interne, ou privé**. On parle d'**encapsulation**. Celles-ci ne doivent pas être utilisées directement par les modules clients (le programme principal, par exemple). Dans les modules, on les signale avec le symbole _ devant la fonction **_nomDeLaFonction**.
Les mises à jour (modifications, améliorations…) des modules doivent rester compatibles avec l'interface.

Convention :	fonction(…) appartient à l'interface.
	_fonction(…) est privée.

En Python, rien n'interdit techniquement de modifier une fonction privée. Il faut s'en abstenir.

Dans notre exemple dates d'anniversaires, on avait limité le nombre de "paquets" à 6 de 64 bits.
Pour être réutilisable dans un autre contexte, le module date devrait pouvoir gérer par lui-même le nombre de paquets, sans incidence sur le fonctionnement du programme principal.
Donc **cree( )** doit créer une structure de taille adaptable, par exemple, et **ajoute(s, x)** doit fonctionner avec cette nouvelle contrainte.

III) <u>EXCEPTIONS</u> :

Prévenir les erreurs d'exécution.
Dans notre exemple, introduire une date négative, non entière ou en dehors de l'intervalle valide crée ou doit créer une erreur avec un message d'erreur impliquant souvent les modules. Résoudre cette erreur demanderait donc à l'utilisateur de connaître la solution choisie pour les fonctions de l'interface. Ça ne doit pas être son problème.
Une exception interrompt le programme. Les modules doivent donc gérer correctement les exceptions.
Il est possible de "lever" *raise* volontairement une exception et de la commenter. NB on a déjà vu une manière avec *assert condition, 'message'*, mais ici il s'agit de gérer une erreur, pas une contrainte ajoutée.

Quelques erreurs : NameError : accès à une variable inexistante.

IndexError : accès à un indice non valide.
KeyError: accès à une clé de dictionnaire non valide.
ZeroDivisionError : division par zéro.
TypeError : opération appliquée à des valeurs incompatibles.

On peut personnaliser ces exceptions, par exemple :

```
if i < 0 :
   raise IndexError ('Indice négatif')
```

On peut préciser l'interface pour aider l'utilisateur :

ajoute(s, x) : ajoute un élément x à la structure s, si 1 < x < 366, sinon lève une exception ValueError ('La date doit être comprise entre 1 et 366').

IV) RATTRAPER UNE EXCEPTION : TRY… EXCEPT... :

Lever une exception conduira à l'interruption du programme si elle se produit.
On peut "rattraper" cette situation. C'est utile pour ne pas planter un programme en cas d'erreur de saisie de l'utilisateur. Il faut lui redonner une chance de saisir une nouvelle entrée.
Voir fonctions saisirDate( ), saisirDate2( ), saisirDate3( ).
Alternatives multiples :

```
try :
  bloc normal
except exception 1:
  bloc alternatif 1
except exception 2:
  bloc alternatif 2
```

Alternative indiscriminée (non recommandé, car cache des erreurs) :
Rattrape tout type d'exception.

```
try :
  bloc normal
except : # sans précision d'exception
  bloc alternatif
```

Documentation : https://pythonbasics.org/try-except/
On peut aussi définir de nouvelles exceptions (par exemple 'HorsBudget'). A revoir après cours sur programmation en classes. C'est une bonne pratique de placer toutes les exceptions dans un module à part.

V) EXERCICES :

Exercice 1 : interface pour les tableaux.
Le langage python intègre l'instruction T[i:j] pour construire un tableau issus de T contenant les éléments i inclus à j exclus, et l'opération T1 + T2 pour concaténer T1 et T2.
Créer votre propre module* qui réalise l'interface **tranche(t, i, j)**, équivalente à T[i:j] et **concat(T1,T2)**, équivalente à T1 + T2, sans utiliser, évidement les fonctions natives T[i:j] ni +, ni la méthode append.
Prévenir des erreurs de saisie comme : T n'est pas un tableau, i et/ou j ne sont pas entiers, ne sont pas compatibles avec les dimensions du tableau. Gérer proprement toutes ces exceptions.
* Pour réaliser à quel point les modules et opérations natives sont bien conçues et pas si simples !

Exercice 2 : tableaux redimensionnables.
Les tableaux de python sont automatiquement redimensionnables, à partir d'une taille de 8 puis par doublement en cas de besoin… Un tableau python est une structure qui contient non seulement les éléments courants, mais aussi la capacité courante, qui se met à jour automatiquement.
Dans cet exercice, un tableau sera représenté par un dictionnaire {'taille' : valeur , 'capacite' : tableau de taille supérieure à n}
Sans utiliser les méthodes existantes pour les tableaux, programmer une interface comprenant les fonctions suivantes :

cree() : crée un tableau vide de taille 0 et de capacité 8.
lit(tableau, i) : renvoie l'élément i du tableau.
ecrit(tableau, i, x) : place x au rang i dans tableau.

ajoute(tableau, x) : ajoute x à la fin du tableau en doublant sa dimension si besoin. Prévoir deux fonctions privées _ajoute(tableau,x) et _double(tableau).

<u>Exercice 3</u> : conception collective d'un mini projet : gestion d'un stock produits et édition de factures avec calcul du montant dû.

Plusieurs groupes se partagent l'implémentation des modules (L'assemblage doit fonctionner).
Le stock de produits est sauvegardé dans stock.csv : **définir les champs utiles.**
Les factures doivent être sauvegardées en fichier texte, sous le nom **facture_num.txt**. num est un numéro entier attribué automatiquement, dans l'ordre croissant.
Définir l'interface du programme principal (les fonctions utilisateurs). Penser aux fonctions d'aide, d'affichage...
Créer tous les modules utiles.